



ANSIBLE FACTS:

- `Ansible facts` are variables that are automatically discovered by Ansible on a managed host.
- Facts contain host-specific information that can be used just like regular variables in plays, conditionals, loops, or any other statement that depends on a value collected from a managed host.
- Some of the facts gathered for a managed host might include:

- The host name
- The kernel version
- The network interfaces
- The IP addresses
- The version of the operating system
- Various environment variables
- The number of CPUs
- The available or free memory
- The available disk space
- Facts are a convenient way to retrieve the state of a managed host and to determine what action to take based on that state.

For example:

- A server can be restarted by a conditional task which is run based on a fact containing the managed host's current kernel version.
- The MySQL configuration file can be customized depending on the available memory reported by a fact.

- The IPv4 address used in a configuration file can be set based on the value of a fact.
 - Normally, every play runs the `setup` module automatically before the first task in order to gather facts. This is reported as the Gathering Facts task in Ansible.
 - By default, you do not need to have a task to run `setup` in your play. It is normally run automatically for you.
 - One way to see what facts are gathered for your managed hosts is to run a short playbook that gathers facts and uses the `debug` module to print the value of the `ansible_facts` variable.

```

```bash
- name: Facts dump

hosts: centnodes

tasks:

 - name: Print all facts

 debug:

 var: ansible_facts
```

```

- The following list shows some facts which might be gathered from a managed node and may be useful in a playbook.

Examples of Ansible Facts:

| Information Needed | Ansible Fact Expression |
|--|--|
| Short host name | <code>ansible_facts['hostname']</code> |
| Fully qualified domain name | <code>ansible_facts['fqdn']</code> |
| Main IPv4 address (default route) | <code>ansible_facts['default_ipv4']['address']</code> |
| List of all network interfaces | <code>ansible_facts['interfaces']</code> |
| Size of <code>/dev/vda1</code> partition | <code>ansible_facts['devices']['vda']['partitions']['vda1']['size']</code> |
| DNS servers list | <code>ansible_facts['dns']['nameservers']</code> |
| Running kernel version | <code>ansible_facts['kernel']</code> |

Remember that when a variable's value is a hash/dictionary, there are two syntaxes that can be used to retrieve the value.

```
```bash
ansible_facts['default_ipv4']['address']
```

can also be written

```bash
ansible_facts.default_ipv4.address
```

```
ansible_facts['dns']['nameservers']
ansible_facts.dns.nameservers
```
```

EXAMPLE:

```
```bash
- hosts: all

tasks:
 - name: Prints various Ansible facts

debug:
 msg: >

 The default IPv4 address of {{ ansible_facts.fqdn }} is {{ ansible_facts.default_ipv4.address }}
```
```

- Before `Ansible 2.5`, facts were injected as individual variables prefixed with the string `ansible_` instead of being part of the `ansible_facts` variable. For example, the `ansible_facts['distribution']` fact would have been called `ansible_distribution`.

- You can turn off the old naming system by setting the `inject_facts_as_vars` parameter in the `[default]` section of the Ansible configuration file to `false`. The default setting is currently `true`.

TURNING OFF FACT GATHERING

- Sometimes, you do not want to gather facts for your play. There are a couple of reasons why this might be the case.
- It might be that you are not using any facts and want to speed up the play or reduce load caused by the play on the managed hosts.
- It might be that the managed hosts cannot run the setup module for some reason, or need to install some prerequisite software before gathering facts.
- To disable fact gathering for a play, set the `gather_facts` keyword to `no`

CREATING CUSTOM FACTS:

- Administrators can create `custom facts` which are stored locally on each managed host.
- These facts are integrated into the list of standard facts gathered by the `setup` module when it runs on the managed host.
- These allow the managed host to provide arbitrary variables to Ansible which can be used to adjust the behavior of plays.
- Custom facts can be defined in a static file, formatted as an `INI` file or using `JSON`.
- They can also be executable scripts which generate JSON output, just like a dynamic inventory script.
- `Custom facts` allow administrators to define certain values for managed hosts, which plays might use to populate configuration files or conditionally run tasks.
- Dynamic custom facts allow the values for these facts, or even which facts are provided, to be determined programmatically when the play is run.
- By default, the `setup module` loads custom facts from files and scripts in each `**managed host's **/etc/ansible/facts.d` directory. The name of each file or script must end in `.fact` in order to be used. Dynamic custom fact scripts must output JSON-formatted facts and must be executable.

EXAMPLE:

```
```bash
[packages]
web_package = httpd
db_package = mariadb-server
[users]
user1 = joel
user2 = peter
```
```

- Custom facts are stored by the `setup` module in the `ansible_facts.ansible_local` variable.
- Facts are organized based on the name of the file that defined them.
- For example, assume that the preceding custom facts are produced by a file saved as `/etc/ansible/facts.d/custom.fact` on the managed host. In that case, the value of `ansible_facts.ansible_local['custom']['users']['user1']` is `joe`.
- Custom facts can be used the same way as default facts in playbooks:

EXAMPLE:

```
```bash
- hosts: all
tasks
- name: Prints various Ansible facts
debug:
msg: >
The package to install on {{ ansible_facts['fqdn'] }}
is {{ ansible_facts['ansible_local']['custom']['packages'] ['web_package'] }}
```
```

USING MAGIC VARIABLES

- Some variables are not facts or configured through the setup module, but are also automatically set by Ansible.
- These magic variables can also be useful to get information specific to a particular managed host.

Four of the most useful are:

hostvars

- Contains the variables for managed hosts, and can be used to get the values for another managed host's variables. It does not include the managed host's facts if they have not yet been gathered for that host.

group_names

- Lists all groups the current managed host is in

groups

- Lists all groups and hosts in the inventory.

inventory_hostname

- Contains the host name for the current managed host as configured in the inventory. This may be different from the host name reported by facts for various reasons.

```
```bash
```

```
ansible localhost -m debug -a 'var=hostvars["localhost"]'
```

```
```
```

EXAMPLES:

```
```bash
```

```
[general]
```

```
package = httpd
```

```
service = httpd
```

```
state = started
```

```
enabled = true
```

```
```
```

PlayBook 1: Install Custom Facts:

```
``yaml
```

```
- name: Install remote facts
```

```
hosts: webserver
```

```
vars:
```

```
  remote_dir: /etc/ansible/facts.d
```

```
  facts_file: custom.fact
```

```
tasks:
```

```
- name: Create the remote directory
```

```
  file:
```

```
    state: directory
```

```
    recurse: yes
```

```
    path: "{{ remote_dir }}"
```

```
- name: Install the new facts
```

```
  copy:
```

```
    src: "{{ facts_file }}"
```

```
    dest: "{{ remote_dir }}"
```

```
``
```

PlayBook_Example- That uses custom facts:

```
``yaml
```

```
- name: Install Apache and starts the service
```

```
hosts: webserver
```

```
tasks:
```

```
- name: Install the required package
```

```
  yum:
```

```
    name: "{{ ansible_facts['ansible_local']['custom']['general'] ['package'] }}"
```

```
    state: latest
```

```
- name: Start the service
```

```
  service:
```

```
    name: "{{ ansible_facts['ansible_local']['custom']['general'] ['service'] }}"
```

```
    state: "{{ ansible_facts['ansible_local']['custom']['general'] ['state'] }}"
```

```
    enabled: "{{ ansible_facts['ansible_local']['custom']['general'] ['enabled'] }}"
```

```
``
```

